

Академија техничко-уметничких
струковних студија Београд

Одсек Висока школа електротехнике и
рачунарства



Примењени истраживачки рад:

**Интегрисана технологија за обраду аудио сигнала
помоћу Arduino микроконтролера и C++ језика**

Професор:

др Перица Штрбац

Студент:

Алекса Матејић РИН-33/17

Београд, јун 2023.

Кандидат: **Алекса Матејић**

Број индекса: **РИН-33/17**

Студијски програм: **Рачунарско инжењерство**

Тема: **Интегрисана технологија за обраду аудио сигнала помоћу Arduino микроконтролера и C++ језика**

Основни задаци:

1. **Опис: Arduino, C++, потребне електронске компоненте и аудио ефеката**
2. **Имплементација хардверског и софтверског дела решења за обраду аудио сигнала**
3. **Приказ експерименталних резултата**

Резиме:

Овај истраживачки рад истражује дизајн и имплементацију програмабилне гитарске педале користећи *Arduino* и *C++* програмски језик. Пројекат обухвата основе аудио *DSP*-а и примену *FFT* алгоритма за постизање пожељних аудио ефеката. Студија такође пружа преглед педала за гитарске ефекте уопште. Методологија укључена у овај пројекат укључује одабир одговарајућих хардверских компоненти, пројектовање и имплементацију софтвера, тестирање и процену перформанси система. Главни допринос овог рада је развој програмабилне гитарске педале са могућношћу генерисања различитих гитарских звукова коришћењем техника дигиталне обраде сигнала. Важност ове теме лежи у растућем интересовању за развој приступачних и лако прилагодљивих педала за гитару у музичкој индустрији.

Кључне речи: хардвер, софтвер, *c++*, *arduino*, дигитална обрада сигнала, педале за гитарске ефекте

Abstract:

This research work explores the design and implementation of a programmable guitar pedal using Arduino and C++ programming language. The project involves the basics of audio DSP and the application of the FFT algorithm to achieve desirable audio effects. The study also provides an overview of guitar effects pedals in general. The methodology involved in this project includes selecting appropriate hardware components, designing and implementing software, testing, and evaluating the performance of the system. The main contribution of this work is the development of the programmable guitar pedal with the capability of generating various guitar sounds by utilizing digital signal processing techniques. The importance of this topic lies in the growing interest in developing affordable and easily customizable guitar pedals in the music industry.

Key words: hardware, software, *c++*, *arduino*, digital signal processing, guitar effects pedals

Садржај

1. Увод.....	1
1.1. Предмет и циљ истраживања.....	1
1.2. Значај теме.....	1
1.3. Методологија.....	1
2. C++ програмски језик.....	2
2.1. Дефиниција и историја C++ језика.....	2
2.2. Значај C++ у програмирању данас.....	3
3. Arduino.....	5
3.1. Дефиниција и историја Arduino.....	6
3.2. Значај Arduino у области електронике и „уради сам“ пројеката.....	6
3.3. Хардвер.....	7
3.4. Arduino UNO.....	7
3.4.1. Напајање.....	9
3.4.2. Пинови.....	9
3.4.3. Дугме за ресет.....	10
3.4.4. LED индикатор напајања.....	10
3.4.5. TX и RX LED индикатори.....	10
3.4.6. Главно интегрисано коло (IC).....	11
3.4.7. Регулатор напона.....	11
3.5. Техничке спецификације.....	12
3.6. Софтвер.....	13
3.6.1. Основи програмирања Arduino.....	14
3.5. Штитови.....	15
4. Ефекти за гитару (гитарске педале).....	17
4.1. Како раде гитарске педале.....	18
4.2. Типови гитарских педала.....	19
4.3. „Уради сам“ гитарске педале.....	19
5. Обрада аудио сигнала помоћу Arduino.....	21
5.1. Типови сигнала.....	21
5.2. DSP употребом Arduino.....	22
5.3. FFT алгоритам.....	22
6. Програмабилна гитарска педала са Arduino.....	26
7. Закључак.....	29
8. Литература.....	30

Свет гитарске музике је одувек био место експериментисања и иновација, са музичарима који непрестано трагају за новим начинима за стварање јединствених звукова. Један од начина да се то постигне је коришћење педала за гитарске ефекте, које су електронска кола која мењају изворни звук електричне гитаре. Последњих година расте интересовање за прављење педала за гитарске ефекте по мери помоћу микроконтролера као што је *Arduino*, који пружају моћну платформу за креирање сложених и разноврсних ефеката.

У овом раду ћемо истражити како да креирамо педале по мери за гитарске ефекте користећи програмски језик *C++* и *Arduino* платформу, уз неопходну пратњу електронских кола. Почећемо тако што ћемо разговарати о основама *C++* језика и *Arduino* платформе, а затим прећи на то како програмирати ефекте по мери за гитаре користећи *C++* на *Arduino*. Такође ћемо истражити хардвер и софтвер потребан за прављење програмабилне гитарске педале са *Arduinom*, укључујући отпорнике, кондензаторе, *LED* диоде и друге компоненте.

1.1. Предмет и циљ истраживања

Како се програмибилна педала за гитару може дизајнирати и имплементирати користећи *Arduino* и *C++* програмски језик, а истовремено укључује аудио *DSP* и *FFT* алгоритам за производњу различитих звукова гитаре?

1.2. Значај теме

Важност ове теме лежи у растућем интересовању музичара и ентузијаста за развој приступачних и лако прилагодљивих педала за гитару. Коришћењем лако доступних хардверских компоненти и програмских језика отвореног кода, овај пројекат има за циљ да обезбеди приступачну платформу за експериментисање са различитим аудио ефектима и генерисање јединствених звукова гитаре. Штавише, нагласак пројекта на дигиталној обради сигнала и *FFT* алгоритму отвара могућности за будућа истраживања у аудио анализи и синтези.

1.3. Методологија

Методологија овог пројекта укључује неколико корака, укључујући:

- Одабир одговарајућих хардверских компоненти као што су *Arduino UNO* плоча, *Audio Shield*, дугмад, потенциометри итд.
- Дизајнирање и имплементација софтвера користећи *C++* програмски језик за контролу аудио сигнала, анализу звучних таласа и генерисање жељених излаза.
- Тестирање система симулацијом различитих ефеката гитаре и верификовањем излаза у односу на очекиване резултате.
- Процена перформанси система у смислу тачности, стабилности и ефикасности.

2. C++ програмски језик

C++ је програмски језик опште намене, високог нивоа који је развијен раних 1980-их као проширење C језика. Дизајниран је за системско програмирање и хардверску манипулацију ниског нивоа, али је временом постао популаран језик за развој широког спектра апликација.

C++ је објектно оријентисан језик, што значи да подржава карактеристике као што су енкапсулација, наслеђивање и полиморфизам. Такође укључује подршку за генеричко програмирање путем шаблона, који омогућавају програмерима да напишу код који се лако може прилагодити за различите типове података.

У погледу синтаксе, C++ је сличан C, али укључује додатне карактеристике као што су класе и именски простори. Такође подржава преоптерећење оператора, шаблоне функција и оператора и руковање изузетцима.

Све у свему, C++ је моћан и свестран језик са широким спектром апликација. Његова подршка за објектно оријентисано програмирање и генеричко програмирање чини га погодним за велике пројекте развоја софтвера, док његове перформансе и могућности ниског нивоа чине га идеалним за системско програмирање и друге задатке где је ефикасност критична.

2.1. Дефиниција и историја C++ језика

C++ је програмски језик опште намене који је развио Бјарне Строуструп 1983. године у *Bell Labs*-у као проширење програмског језика C. Циљ C++ је био да обезбеди ефикаснији и моћнији језик уз одржавање компатибилности са постојећим C кодом. Строуструп је у то време радио на својој докторској тези у *Bell Labs*-у и желео је да створи језик који би могао да се користи за развој софтверских система великих размера, а да је и даље брз и ефикасан.

Једна од кључних карактеристика C++-а је његова подршка објектно оријентисаном програмирању (ООП). ООП је програмска парадигма која се фокусира на идеју објеката, који су инстанце класа које обухватају податке и функције. C++ такође подржава друге парадигме програмирања као што су процедурално програмирање и генеричко програмирање.

C++ је објектно оријентисани програмски језик који подржава инкапсулацију, наслеђивање и полиморфизам. То је језик који се може користити за креирање апликација високих перформанси за различите платформе, укључујући десктоп рачунаре, мобилне уређаје и уграђене системе.

C++ је компајлирани језик, што значи да код написан у C++ мора бити преведен у машински код пре него што се може извршити. Ово чини C++ програме веома брзим и ефикасним у поређењу са интерпретираним језицима као што су *Python* или *JavaScript*.

Током година, C++ је наставио да се развија са новим функцијама и побољшањима. Године 1998. објављен је први међународни стандард за C++, познат као C++98 или *ISO/IEC 14882:1998*. Овај стандард је дефинисао многе језичке

карактеристике које су и данас у употреби, као што су шаблони, изузеци и простори имена, што га је учинило још разноврснијим и моћнијим. Данас се C++ широко користи у индустријама као што су игре, финансије и научно рачунарство, између осталих.

Од тада је објављено неколико нових стандарда за C++, укључујући C++11, C++14, C++17 и недавно C++20. Сваки нови стандард је додао нове карактеристике и могућности језику, а истовремено је побољшао перформансе и олакшао писање исправног и ефикасног кода.

Данас се C++ још увек широко користи у многим индустријама за прављење апликација високих перформанси, посебно у областима као што су игре, финансије и научно рачунарство. Док су новији језици као што су *Python* и *JavaScript* постали популарни за веб (енг. *Web*) развој и друге апликације, C++ остаје важан језик за програмирање на нивоу система и рачунарство високих перформанси.

Неке значајне апликације направљене са C++ укључују оперативни систем *Windows*, *Adobe Photoshop* и *MySQL* систем за управљање базом података.

Укратко, C++ је моћан програмски језик који се надограђује на основу C-а и додаје нове могућности за подршку савременим потребама развоја софтвера. Његова свестраност и ефикасност чине га популарним међу програмерима за широк спектар апликација.

2.2. Значај C++ у програмирању данас

C++ је програмски језик који постоји већ неколико деценија и остаје важан део модерног програмирања. Ево неколико разлога зашто је C++ и данас важан језик:

Високе перформансе: Једна од највећих предности C++-а је његова брзина и ефикасност. C++ код се може компајлирати директно у машински код, што му омогућава да ради много брже од интерпретираних језика као што су *Python* или *JavaScript*.

Системско програмирање: C++ је првобитно дизајниран за системско програмирање. То је идеалан језик за креирање оперативних система, драјвера уређаја и другог софтвера ниског нивоа који је у директној интеракцији са хардвером.

Објектно оријентисано програмирање (ООП): C++ је објектно оријентисани језик, што значи да подржава карактеристике као што су енкапсулација, наслеђивање и полиморфизам. Ово олакшава писање сложених програма и управљање великим кодним базама.

Користи се у великим пројектима: Многи велики софтверски пројекти, као што су видео игре и платформе за финансијско трговање, написани су на C++. Његове перформансе и способност да рукује великим кодним базама чине га атрактивним избором за ове типове пројеката.

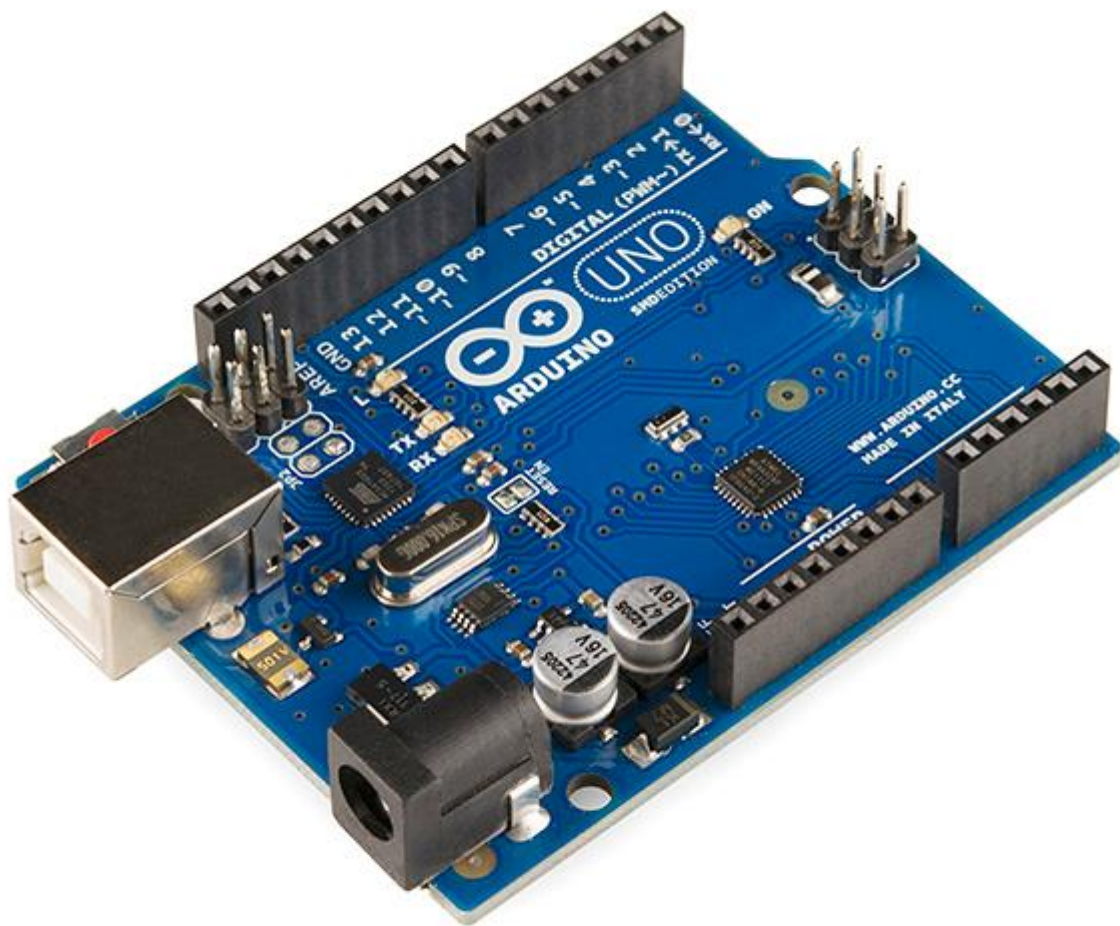
Компатибилност међу платформама: C++ код се може компајлирати за рад на различитим платформама, укључујући *Windows*, *Mac OS*, *Linux*, па чак и уграђене системе.

Доступне библиотеке: C++ има велики број доступних библиотека, што омогућава програмерима да лако приступе унапред написаном коду и избегну поновно измишљање тачка.

Све у свему, C++ остаје важан језик у програмирању данас због својих високих перформанси, подршке за ООП, способности руковања великим базама кода, компатибилности међу платформама и доступности библиотека.

3. Arduino

Arduino је платформа отвореног кода која се користи за изградњу електронских пројеката. Састоји се од хардверских и софтверских компоненти које омогућавају корисницима да креирају широк спектар интерактивних уређаја и прототипова. *Arduino* плоча је мала плоча са микроконтролером која се може програмирати помоћу *Arduino* интегрисаног развојног окружења (*IDE*). Ово омогућава корисницима да напишу код и поставе га на плочу, која затим може да ступи у интеракцију са различитим сензорима и актуаторима. *Arduino* је постао популаран међу хобистима, едукаторима и професионалцима због ниске цене, лакоће употребе и свестраности. Обично се користи у пројектима који укључују роботiku, кућну аутоматизацију, носиву технологију и још много тога.



Слика 3.1 Arduino

3.1. Дефиниција и историја *Arduina*

Arduino је електронска платформа отвореног кода заснована на хардверу и софтверу који се лако користи. Састоји се од програмабилне плоче микроконтролера или „*Arduino board*“ и окружења за развој софтвера које ради на вашем рачунару. *Arduino* плоча се може повезати са широким спектром сензора и актуатора, омогућавајући вам да креирате све врсте интерактивних електронских пројеката.

Arduino је 2005. године креирао тим истраживача у Италији, на челу са Масимом Банзијем, који је желео да створи платформу лаку за коришћење за уметнике, дизајнере и хобисте за изградњу интерактивних електронских пројеката. Били су фрустрирани сложеностју постојећих платформи микроконтролера, које су захтевале познавање програмских језика ниског нивоа и опсежну експертизу у области електронике.

Да би решио овај проблем, *Arduino* тим је креирао једноставну платформу прилагођену кориснику која се може програмирати коришћењем поједностављене верзије програмског језика C++. Такође су развили окружење за развој софтвера отвореног кода које је олакшало писање и постављање кода на плочу, као и велику заједницу корисника који су своје пројекте и идеје делили на мрежи.

Данас је *Arduino* постао једна од најпопуларнијих електронских платформи на свету, коју користе подједнако хобисти, студенти и професионалци. Његова популарност је због његове једноставности, свестраности и приступачности, као и због јаке заједнице корисника који доприносе његовом развоју и деле своје пројекте и идеје са другима.

3.2. Значај *Arduina* у области електронике и „уради сам“ пројеката

Arduino платформа је постала веома популарна у области електронике и „уради сам“ пројеката због своје свестраности, лакоће коришћења и ниске цене. Неке од кључних предности коришћења *Arduina* у пројектима електронике укључују:

Приступачност: Платформу је лако научити и користити, што је чини доступном и почетницима и стручњацима.

Флексибилност: *Arduino* плоче се могу користити за широк спектар апликација и лако се програмирати за обављање различитих задатака.

Интерактивност: *Arduino* плоче могу да комуницирају са физичким светом преко сензора, актуатора и других електронских компоненти.

Ниска цена: *Arduino* плоче су релативно јефтине и могу се користити у различитим пројектима без губитка новца.

Отворени код: *Arduino* платформа је отвореног кода, што значи да свако може приступити датотекама дизајна и модификовати их тако да одговарају својим потребама.

Све у свему, *Arduino* је имао значајан утицај на област електронике и „уради сам“ пројеката, чинећи га лакшим и доступнијим хобистима, студентима и професионалцима да креирају иновативне пројекте и решења.

3.3. Хардвер

Arduino је хардвер отвореног типа. Хардверски референтни дизајни се дистрибуирају под лиценцом *Creative Commons Attribution Share-Alike 2.5* и доступни су на *Arduino* веб локацији. Доступне су и датотеке распореда и производње за неке верзије хардвера.

Иако су хардверски и софтверски дизајни слободно доступни под *copyleft* лиценцама, програмери су затражили да име *Arduino* буде ексклузивно за званични производ и да се не користи за изведена дела без дозволе.

Већина *Arduino* плоча састоји се од *Atmel 8-bit AVR* микроконтролера са различитим количинама флеш меморије, пинова и функција. Плоче користе једноредне или дворедне игле, илити пинови, или женске заглавља које олакшавају везе за програмирање и уградњу у друга кола. Они се могу повезати са додатним модулима који се називају штитови. Вишеструки и евентуално наслагани штитови могу се појединачно адресирати преко *I2C* серијске магистрале. Већина плоча укључује линеарни регулатор од 5 V и кристални осцилатор од 16 MHz или керамички резонатор.

Arduino микроконтролери су унапред програмирани са програмом за покретање који поједностављује учитавање програма у флеш меморију на чипу. Плоче се учитавају програмским кодом преко серијске везе са другим рачунаром. Тренутне *Arduino* плоче су програмиране преко универзалне серијске магистрале (*USB*).

Arduino плоча излаже већину *I/O* пинова микроконтролера за употребу од стране других кола. Неки модели *Arduino*-а, укључујући *Uno*, обезбеђују 14 дигиталних *I/O* пинова, од којих шест могу да производе модулисане сигнале ширине импулса и шест аналогних улаза, који се такође могу користити као шест дигиталних *I/O* пинова. Ови пинови се налазе на врху плоче, преко женских заглавља од 0,1 инча (2,54 mm). Неколико додатака за апликације штитова је такође комерцијално доступно.

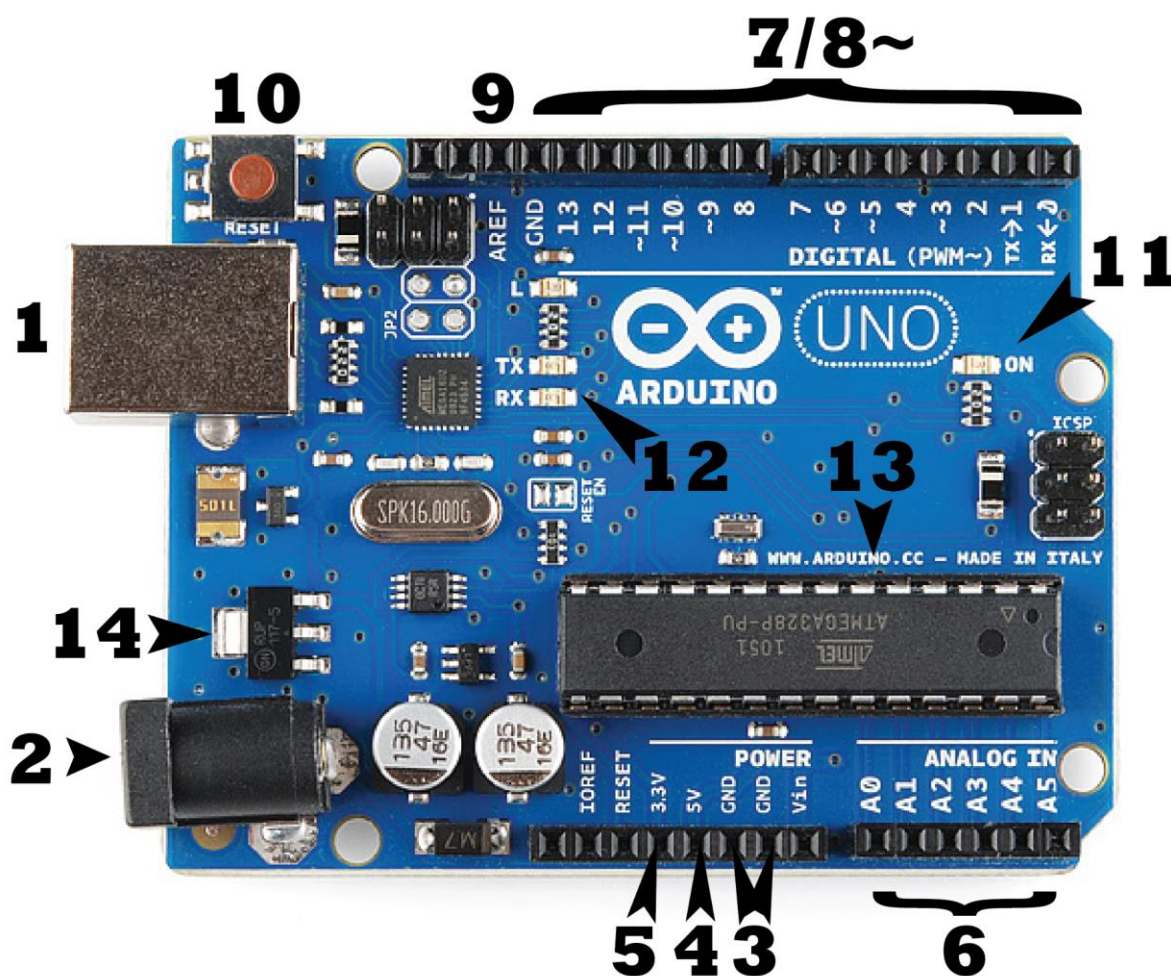
Постоје многе *Arduino* компатибилне и *Arduino* изведене плоче. Неки су функционално еквивалентни *Arduinu* и могу се користити наизменично. Многи побољшавају основни *Arduino* додавањем излазних драјвера, често за употребу у школском образовању, како би се поједноставила прављење колица и малих робота. Други су електрични еквивалентни, али мењају фактор форме, понекад задржавајући компатибилност са штитовима, понекад не. Неке варијанте користе различите процесоре, различите компатибилности.

3.4. Arduino UNO

Arduino UNO је плоча микроконтролера заснована на *ATmega328P* (табелу са подацима). Има 14 дигиталних улазно/излазних пинова (од којих се 6 може користити као *PWM* излази), 6 аналогних улаза, керамички резонатор од 16 MHz (*CSTCE16M0V53-R0*), *USB* конекцију, прикључак за напајање, *ICSP* заглавље и дугме за ресетовање. Садржи све што је потребно за подршку микроконтролеру; једноставно га повежите са рачунаром помоћу *USB* кабла или га напајајте АС-на-DC адаптером или батеријом да бисте започели. Можете да петљате са својим *UNO*-ом без превише бриге о томе да ли

ћете учинити нешто погрешно, у најгорем случају можете заменити чип за неколико долара и почни изнова.

„Uno“ значи један на италијанском и изабран је да означи издање *Arduino* софтвера (IDE) 1.0. Uno плоча и верзија 1.0 *Arduino* софтвера (IDE) биле су референтне верзије *Arduina*, које су сада еволуирале у новија издања. Uno плоча је прва у низу *USB Arduino* плоча и референтни модел за *Arduino* платформу.



Слика 3.2 *Arduino UNO*

3.4.1. Напајање

Свака *Arduino* плоча треба начин да се повеже са извором напајања. *Arduino UNO* се може напајати преко *USB* кабла који долази са вашег рачунара или зидног напајања

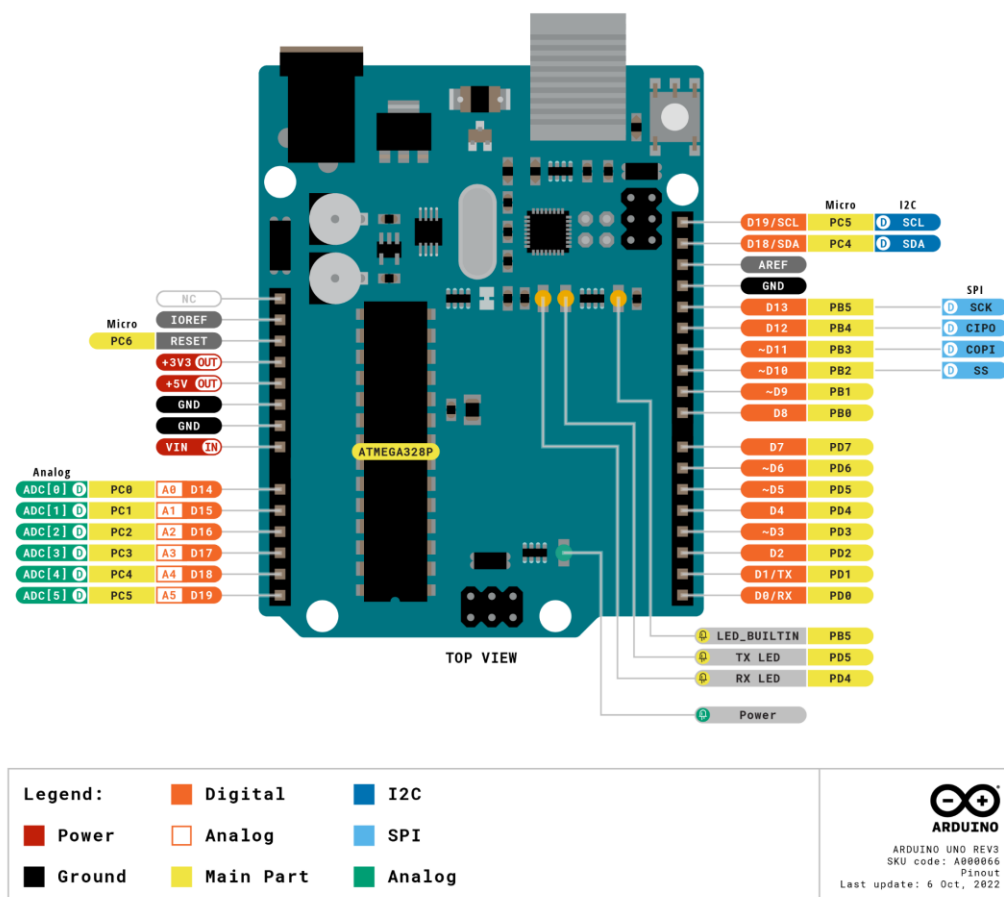
(као што је овај) који се завршава утичницом. На слици 3.2 изнад *USB* прикључак је означен (1), а утичница за цев (енг. *Barrel Jack*) је означена (2).

USB веза је такође начин на који ћете учитати код на вашу *Arduino* плочу. Више о томе како програмирати помоћу *Arduino* можете пронаћи у даљем тексту где ћемо се дотаћи и програмирања *Arduino*.

НАПОМЕНА: Немојте користити напајање веће од 20 волти јер ћете надјачати (и тиме уништити) свој *Arduino*. Препоручени напон за већину *Arduino* модела је између 6 и 12 волти.

3.4.2. Пинови

Пинови на вашем *Arduino* су места на којима повезујете жице да бисте конструисали коло (вероватно у комбинацији са матичном плочом и неком жицом. Обично имају црна пластична заглавља који вам омогућавају да само прикључите жицу директно у плочу. *Arduino* има неколико различитих врста пинова, од којих је свака означена на плочи и користи се за различите функције.



Слика 3.3 *Arduino UNO* пинови

GND, на слици изнад означен (3): Скраћеница за „земљу“. Постоји неколико *GND* пинова на *Arduino*, од којих се сваки може користити за уземљење вашег кола.

5V, на слици изнад означен (4) и **3,3V** на слици изнад означен (5): Као што можете претпоставити, пин од 5V даје 5 волти напајања, а пин од 3,3V даје 3,3 волта напајања. Већина једноставних компоненти које се користе са *Arduinom* раде са напоном од 5 или 3,3 волта.

Аналогни, на слици изнад означен (6): Подручје пинова испод ознаке „Аналогни улаз“ (A0 до A5 на *Arduino UNO*) су пинови за аналогни улаз. Ови пинови могу прочитати сигнал са аналогног сензора (попут сензора температуре) и претворити га у дигиталну вредност коју можемо прочитати.

Дигитални, на слици изнад означен (7): Преко пута аналогних пинова налазе се дигитални пинови (0 до 13 на *Arduino UNO*). Ови пинови се могу користити и за дигитални улаз (као што је откривање да ли је дугме притиснуто) и дигитални излаз (као што је напајање *LED*-а).

PWM, на слици изнад означен (8): Можда сте приметили тилду (~) поред неких дигиталних пинова (3, 5, 6, 9, 10 и 11 на *Arduino Uno*). Ови пинови делују као нормални дигитални пинови, али се такође могу користити за нешто што се зове пулсно-ширинска модулација (*PWM*). Имамо туторијал о *PWM*-у, али за сада, размислите о овим пиновима као о могућностима симулације аналогног излаза (као што је бледење *LED*-а у и ван).

AREF, на слици изнад означен (9): Означава аналогну референцу. Већину времена можете оставити овај пин на миру. Понекад се користи за постављање спољног референтног напона (између 0 и 5 волти) као горње границе за пинове аналогног улаза.

3.4.3. Дугме за ресет

Баш као и оригинални Nintendo, *Arduino* има дугме за ресетовање, на слици изнад означен (10). Притиском на њега привремено ће се пин за ресетовање повезати са уземљењем и поново покренути било који код који је учитан на *Arduino*. Ово може бити веома корисно ако се ваш код не понавља, али желите да га тестирате више пута. Међутим, за разлику од оригиналног Nintendo, дување на *Arduino* обично не решава проблеме.

3.4.4. LED индикатор напајања

Одмах испод и десно од речи „*UNO*“ на вашој плочи, налази се сићушна *LED* лампица поред речи „УКЉУЧЕНО“, на слици изнад означен (11). Ова *LED* лампица треба да се упали сваки пут када свој *Arduino* укључите у извор напајања. Ако се ово светло не упали, постоји велика шанса да нешто није у реду.

3.4.5. TX и RX LED индикатори

TX је скраћеница за пренос, *RX* је скраћеница за пријем. Ове ознаке се прилично појављују у електроници да би означиле пинове одговорне за серијску комуникацију. У нашем случају, постоје два места на *Arduino UNO* где се појављују *TX* и *RX* - једном преко дигиталних пинова 0 и 1, а други пут поред *LED* индикатора *TX* и *RX*, на слици изнад означен (12). Ове *LED* диоде ће нам дати неке лепе визуелне индикације кад год наш *Arduino* прима или преноси податке (као када учитавамо нови програм на плочу).

3.4.6. Главно интегрисано коло (IC)

Црна ствар са свим металним ногама је *IC*, или интегрисано коло, на слици изнад означен (13). Замислите то као мозак нашег *Arduina*. Главно *IC* на *Arduinu* се мало разликује од једног до другог типа плоче, али је обично из *ATmega* линије *IC*-а компаније *ATMEL*. Ово може бити важно, јер ћете можда морати да знате тип *IC*-а (заједно са типом плоче) пре него што учитате нови програм из *Arduino* софтвера. Ове информације се обично могу наћи у писаној форми на горњој страни *IC*-а. Ако желите да сазнате више о разлици између различитих *IC*-а, читање листова са подацима је често добра идеја.

3.4.7. Регулатор напона

Регулатор напона, на слици изнад означен (14) заправо није нешто са чиме можете (или би требало) да комуницирате на *Arduinu*. Али потенцијално је корисно знати да је ту и чему служи. Регулатор напона ради тачно оно што каже - контролише количину напона који се пушта у *Arduino* плочу. Замислите то као неку врсту вратара; то ће искључити додатни напон који би могао оштетити коло. Наравно, то има своја ограничења, тако да не прикључујте свој *Arduino* на нешто веће од 20 волти.

3.5. Техничке спецификације

Board	Name	Arduino UNO R3
	SKU	A000066
Microcontroller	ATmega328P	
USB connector	USB-B	
Pins	Built-in LED Pin	13
	Digital I/O Pins	14
	Analog input pins	6
	PWM pins	6
Communication	UART	Yes
	I2C	Yes
	SPI	Yes
Power	I/O Voltage	5V
	Input voltage (nominal)	7-12V
	DC Current per I/O Pin	20 mA
	Power Supply Connector	Barrel Plug
Clock speed	Main Processor	ATmega328P 16 MHz
	USB-Serial Processor	ATmega16U2 16 MHz
Memory	ATmega328P	2KB SRAM, 32KB FLASH, 1KB EEPROM
Dimensions	Weight	25 g
	Width	53.4 mm
	Length	68.6 mm

Слика 3.4 Техничке спецификације

3.6. Софтвер

Arduino софтвер, такође познат као Интегрисано развојно окружење (*IDE*), је апликација за више платформи која вам омогућава да пишете, компајлирате и отпремате код на *Arduino* плоче. То је платформа отвореног кода заснована на софтверу који је једноставан за коришћење.

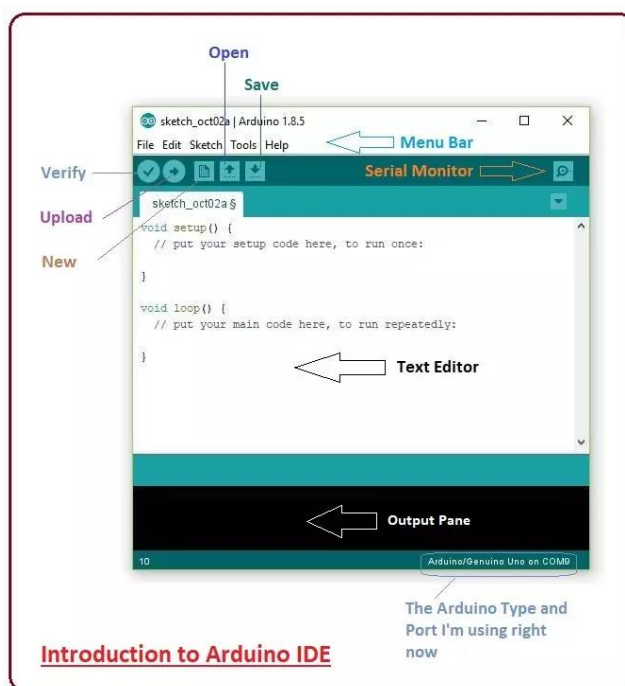
Arduino IDE је написан на Јави (енг. *Java*) и компатибилан је са *Windows*, *macOS* и *Linux* оперативним системима. Пружа интуитиван интерфејс и уређивач прилагођен кориснику који поједностављује процес програмирања наглашавајући синтаксичке грешке, сугеришући завршетак кода и нуди библиотеку унапред написаних примера кода.

Arduino језик је заснован на *Wiring*, поједностављеној верзији C++ која почетницима олакшава учење. Језик пружа скуп функција и библиотека које вам омогућавају интеракцију са хардвером плоче, као што је читање аналогних и дигиталних улаза, контрола *LED* диода и комуникација преко серијских портова.

Када напишете свој код, можете га компајлирати у машински читљива упутства која се могу учитати на *Arduino* плочу. *IDE* укључује уграђени серијски монитор који вам омогућава да видите излаз вашег кода и отклоните све проблеме који се појаве.

Поред тога, *Arduino* подржава широк спектар библиотека и штитова трећих страна који проширују функционалност платформе. Ове библиотеке пружају додатне функције, као што су *Bluetooth* повезивање, контрола мотора и интеграција сензора.

Све у свему, *Arduino* софтвер је моћан алат за учење и израду прототипова електронских пројеката. Његова једноставност и флексибилност чине га доступним почетницима док и даље пружа напредне функције за искусније кориснике.



Слика 3.4 *Arduino UNO* интегрисано развојно окружење (*IDE*)

3.6.1. Основи програмирања *Arduino*

Да бисте започели са *Arduino* програмирањем, потребна вам је *Arduino* плоча, USB кабл и *Arduino* интегрисано развојно окружење (*IDE*). *IDE* је програмски алат који се користи за писање, слика 3.4, компајлирање и отпремање кода на *Arduino* плочу.

Структура *Arduino* програма: *Arduino* програми су написани у C++ програмском језику, иако корисници треба да дефинишу само две функције да би креирали извршни програм. *Arduino* програм, који се назива и скица (енг. *sketch*), структуриран је на следећи начин:

setup(): функција се позива једном када се програм покрене и користи се за иницијализацију различитих компоненти и променљивих подешавања.

loop(): функција ради узастопно док се програм не заустави или док се плоча не искључи. Садржи главну логику програма.

Sketch језик је заснован на C++ програмском језику и лако га је научити, чак и за неког ко је почетник. Скице су у суштини мали програми који раде на *Arduino* плочи и контролишу њене различите функције.

Развојно окружење *Sketch* пружа кориснички интерфејс за креирање и уређивање скица. Поседује уређивач текста, библиотеку уграђених функција и алате за састављање и отпремање скица на вашу *Arduino* плочу.

Као што смо већ рекли, синтакса за *Arduino* програм је слична C++. Ево неких основних команди које се обично користе у *Arduino* програмирању:

pinMode(): користи се за подешавање режима пина (улаз/излаз).

digitalWrite(): користи се за уписивање дигиталне вредности (HIGH/LOW) на пин.

analogRead(): користи се за читање аналогне вредности са пина.

delay(): користи се за паузирање програма на одређено време.

Следи приказ једноставног примера *Arduino* програма којим је микроконтролер испрограмиран тако да трепери *LED* диода спојена на пин 13. У *Arduino* окружењу, корисник би могао да напише програм овако:

```
#define LED_PIN 13

void setup () {
    pinMode (LED_PIN, OUTPUT); // definiši pin 13 kao
digitalni izlaz
}

void loop () {
    digitalWrite (LED_PIN, HIGH); // укључи LED
    delay (1000); // саčekaj jedan sekund (1000
milisekundi)
    digitalWrite (LED_PIN, LOW); // искључи LED
    delay (1000); // саčekaj jedan sekund
}
```

Блок кода 1.1 Пример типичног првог програма за Arduino

Да би овај код исправно радио, анода *LED*-а мора бити повезана на пин 13, а катода *LED*-а мора бити спојена на масу (у овом случају негативни део напајања). Овај пример стандардни C++ компајлер не би видео као исправан програм, међутим, када корисник изда команду за превођење, овај код се додаје са одређеним заглављем и једноставном функцијом `main()`, што га чини исправним.

Arduino програмирање је одличан начин да научите о електроници и програмирању. Са основним разумевањем структуре и синтаксе *Arduino* програма, можете почети да креирате сопствене пројекте и експериментишете са различитим сензорима и компонентама.

3.5. Штитови

Штит (енг. *shield*) за *Arduino* је у суштини плоча за проширење која се лако може повезати са *Arduinom* и дизајнирана је да стане на врх *Arduino* плоче, пружајући додатне хардверске карактеристике и функционалност које се могу користити у различитим пројектима. Сврха штита је да поједностави процес интеграције додатних функција у *Arduino* пројекте, обезбеђујући унапред изграђене модуле који се могу лако повезати на главну плочу. Штит је добио име по чињеници да је дизајниран да "заштити" основну *Arduino* плочу, штитећи је од оштећења, а истовремено додаје нове могућности.

Штитови су популарни међу хобистима и произвођачима јер пружају једноставан начин за додавање нових функција *Arduino* пројекту, без потребе за додатним радом или стручношћу у дизајну електронике. Многи штитови су унапред изграђени модули који укључују сензоре, дисплеје, бежичне комуникационе модуле и друге хардверске компоненте које се могу користити у различитим пројектима. Једноставним укључивањем штита на врх *Arduino* плоче, програмери могу брзо да додају нове функције, као што су сензори температуре, бежично повезивање или контрола мотора.

Штитови могу доћи у много различитих облика, у зависности од специфичне функционалности коју пружају. Неки штитови могу да додају бежичну везу, као што су *Wi-Fi* или *Bluetooth*, док други могу да садрже сензоре за мерење температуре, влажности или других фактора околине. Постоје и штитови који укључују драјвере (енг. *driver*) мотора, *LCD* екране, *GPS* пријемнике и још много тога.

Један пример популарног штита је *Ethernet Shield*, који пружа једноставан начин за повезивање *Arduino* плоче са локалном мрежом или интернетом. Штит укључује чип за *Ethernet* контролер и *PJ45* прикључак, омогућавајући програмерима да шаљу и примају податке преко жичане везе. Други примери штитова укључују *GPS* штитове, који пружају функцију праћења локације, и штитове возача мотора, који омогућавају прецизну контролу *DC* мотора или корачних мотора.

Коришћење штита је обично једноставан процес. Када се штит прикључи на *Arduino* плочу, може се програмирати помоћу стандардног *Arduino* интегрисаног развојног окружења (*IDE*), као и сваки други *Arduino* пројекат. Ово омогућава програмерима да брзо додају нове функције својим пројектима без потребе да брину о дизајнирању сложених кола или ручном повезивању. Штитови често долазе са библиотекама и примерима кода који олакшавају почетак коришћења нове функционалности коју пружа штит.

Укратко, штит за *Arduino* је плоча за проширење која пружа једноставан начин за додавање новог хардвера и додатне функционалности у *Arduino* пројекат, без потребе за стручношћу у дизајну електронике или сложеним ожичењем, што олакшава интеграцију нових функција у *Arduino* пројекте. Штитови могу укључивати различите хардверске компоненте и дизајнирани су да се лако повезују на *Arduino* плочу, долазе у много различитих облика и могу се програмирати помоћу стандардног *Arduino IDE*-а. Касније ћемо видети како се овакви штитови могу користити код процесирања аудио сигнала.

4. Ефекти за гитару (гитарске педале)

Педале за гитарске ефекте су електронски уређаји који мењају звук електричне гитаре. Обично су смештени у малој кутији која седи на поду и повезана је са гитаром и појачалом кабловима.

Свака педала обично има специфичан ефекат или комбинацију ефеката, као што су дисторзија, овердриве, кашњење, реверб, хор, фејзер, флангер, вах и још много тога. Гитаристи користе ове педале да обликују свој тон и креирају различите звукове за различите музичке жанрове, стилове и расположења.

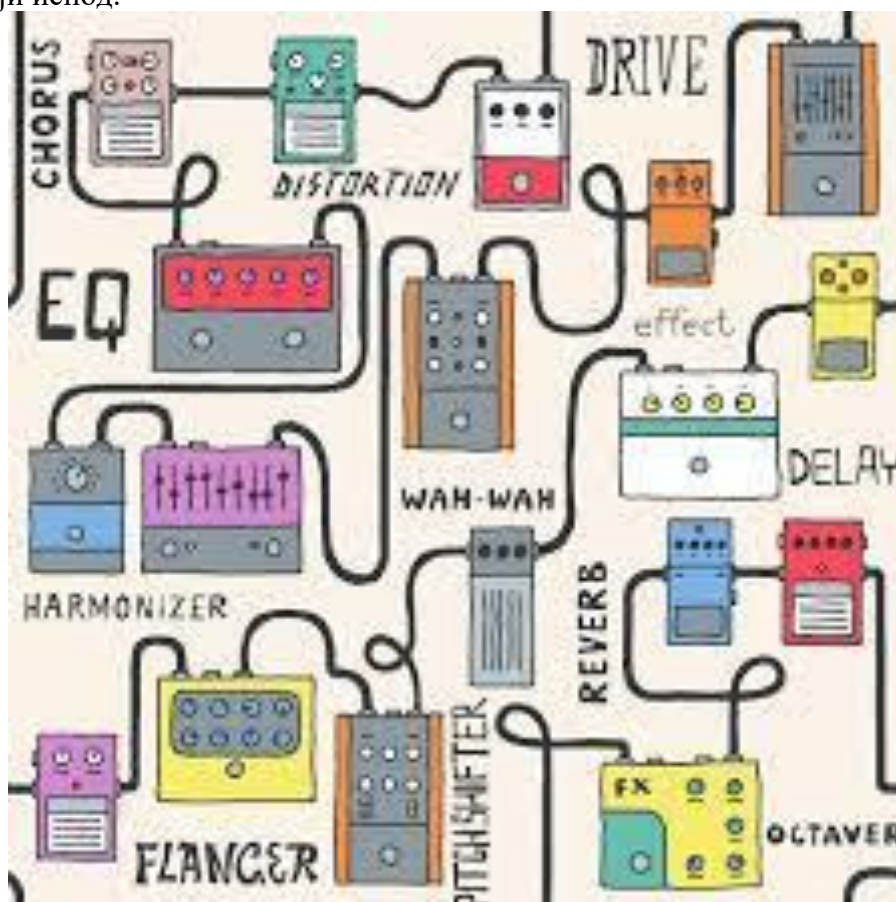


Слика 4.1 Ibanez TS9 Tube Screamer, најпопуларнија овердриве гитарска педала

На пример, педала за дисторзију може да дода ритам, шкрипање и одржавање звуку гитаре, савршено за хеви метал и хард рок. Педала за одлагање може да створи одјеке, понављања и повратне информације, одлично за амбијенталну и психоделичну

музику. Вах педала може да филтрира и поништи тон, популаризован од стране фанк и рок гитариста.

Педале за гитарске ефекте се могу користити појединачно или у комбинацији са другим педалама за постизање сложенијих и нијансираних звукова. Такође се могу поставити пре или после других педала у сигналном ланцу, утичући на укупан тон и динамику. Колико далеко у комбиновању се може стићи, можете погледати на илустрацији испод:



Слика 4.2 пример уланчаних гитарских педала

Све у свему, педале за гитару су суштински алат за гитаристе који желе да се изразе кроз музику и истраже нове звучне могућности.

4.1. Како раде гитарске педале

Генерално, гитарске педале функционишу тако што модификују сигнал гитаре пре него што стигне до појачала или друге опреме за обраду. Ово се може постићи на различите начине, као што су аналогна или дигитална кола за обраду сигнала, филтери, осцилатори, или чак физички механизми као што су опруге.

На пример, педала за дисторзију ради тако што појачава сигнал гитаре све док се не исече или изобличи, стварајући груби, изобличени тон који је популаран у рок музици. Педала за одлагање ради тако што снима и понавља сигнал гитаре након кратког одлагања, стварајући ефекат еха.

Већина педала има неколико дугмади или прекидача који омогућавају кориснику да подеси количину и врсту ефекта примењеног на сигнал, као и друге параметре као што су напад, опадање, фреквентни одзив и још много тога.

4.2. Типови гитарских педала

Постоји много типова педала за гитарске ефекте, од којих је свака дизајнирана да створи одређени звук или промени тон ваше гитаре на специфичан начин. Ево неких уобичајених типова педала за гитарске ефекте:

Педале за изобличење (енг. *Distortion pedals*): Ове педале додају изобличење сигналу ваше гитаре, стварајући оштар, преоптерећен звук који се обично користи у рок и метал музици.

Овердрајв педале (енг. *Overdrive pedals*): Слично као и педале за дисторзију, педале за Овердрајв додају топлу шкрипу тону ваше гитаре, али са мање одрживости и компресије од педала за дисторзију.

Зујне педале (енг. *Fuzz pedals*): Зујне педале производе нејасан, зујан звук који се често повезује са класичним роком и психоделичном музиком.

Педале за одлагање (енг. *Delay pedals*): Педале за одлагање понављају сигнал ваше гитаре у одређеном интервалу, стварајући ефекат еха који може да варира од суптилног до драматичног.

Амбијенталне педале (енг. *Reverb pedals*): Амбијенталне педале додају осећај простора тону ваше гитаре, симулирајући звук различитих акустичних окружења као што су собе, дворане и катедрале.

Хорске педале (енг. *Chorus pedals*): Хорске педале стварају светлуцави, удвостручени ефекат додавањем мало одложене, модулиране копије сигнала ваше гитаре оригиналу.

Вах педале (енг. *Wah pedals*): Вах педале стварају широк, резонантни ефекат селективним филтрирањем одређених фреквенција сигнала ваше гитаре.

Ово је само неколико најпопуларнијих примера многих врста доступних педала за гитарске ефекте. Свака врста педала може имати свој јединствени звук и може се комбиновати са другим педалама како би се створили још сложенији ефекти.

4.3. „Уради сам“ гитарске педале

Данас је све више популарнија метода „Уради сам“ (енг. *DIY*) прављења педала за гитарске ефекте, те може бити забаван и користан хоби за гитаристе који су заинтересовани да прилагоде свој звук. Покушаћемо у овом раду да приближимо како би то изгледало ако бисмо искористили микроконтролер *Arduino* којим бисмо програмибилним путем измењивали изворни звук гитаре на начине како то раде гитарске педале за ефекте.

Прављење сопствене педале за гитарске ефекте „уради сам“ може бити одличан начин да добијете управо звук који тражите, а истовремено научите више о електроници

и њеном односу са музиком. Доступни су многи ресурси за прављење педала за гитарске ефекте, укључујући онлајн (енг. *online*) туторијале, књиге и комплете који долазе са свим потребним компонентама.

Једна важна ствар коју треба имати на уму када правите сопствену педалу за гитарске ефекте је да безбедност увек треба да буде на првом месту. Радићете на струју, тако да је важно да предумете одговарајуће мере предострожности и пажљиво пратите сва упутства. Поред тога, можда ће бити потребно неко знање о електроници и лемљењу, па се уверите да сте задовољни овим вештинама пре него што започнете свој пројекат.



Слика 4.3 пример „уради сам“ гитарске педале

Све у свему, „уради сам“ педале за гитарске ефекте могу бити одличан начин да прилагодите свој звук и истовремено научите више о електроници. Уз праве ресурсе и мало стрпљења, можете направити јединствену и персонализовану педалу која ће вам помоћи да постигнете звук о којем сте сањали.

5. Обрада аудио сигнала помоћу Arduinoa

Обрада сигнала се односи на било коју врсту модификације или анализе сигнала. Ове технике обраде се користе за побољшање ефикасности система. Обрада сигнала има

примену у скоро свим областима живота, али пре него што уђемо у то, хајде да дефинишемо сигнал.

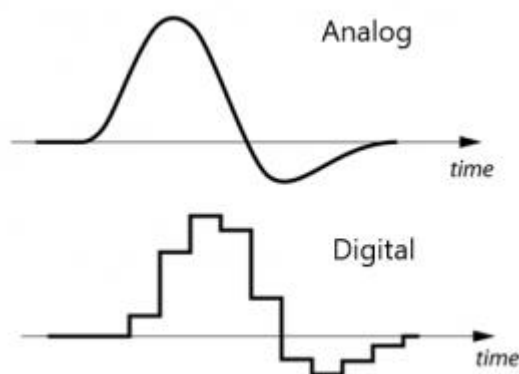
Сигнал је електрични импулс или талас који носи информацију. Ово се односи на променљиве струје, или напоне, или електромагнетне таласе који преносе податке у било којој тачки у електричним системима. Хајде да укратко прођемо кроз његове врсте.

5.1. Типови сигнала

У основи, постоје две врсте сигнала:

Аналогни: Аналогни сигнал је континуирани временски променљив сигнал који има бесконачну количину узорака у себи. Ови аналогни сигнали се даље деле на композитне и једноставне аналогне сигнале. Синусоидни таласи су једноставни аналогни сигнали, а сви остали, композитни сигнали, настају комбинацијом више варијанти једноставних синусних таласа.

Дигитални: Дигитални сигнали такође носе податке као и аналогни сигнали, али у случају дигиталних сигнала, ове вредности су дискретне. Представља податке у облику дискретног низа вредности и дефинисан је интервалом или брзином пријеноса. Дигитални сигнали стога садрже скуп бројева који описују узорке континуалне променљиве.



Слика 5.1 типови сигнала

Слика изнад (5.1) представља исти сигнал у аналогном и дигиталном облику. Имајући на уму да ивице у дигиталном сигналу представљају тачку у којој је узорак узет.

Сходно типовима сигнала, постоје аналогна обрада сигнала и дигитална обрада сигнала. У овом раду ћемо се бавити дигиталном обрадом сигнала, уз помоћ .

5.2. DSP употребом Arduino

Дигитална обрада сигнала (енг. *Digital Signal Processing*, или скраћено *DSP*) је метода анализе и манипулације сигнаlima како би се из њих извукле корисне информације. Са појавом моћних микропроцесора као што је *Arduino*, постало је могуће имплементирати сложене *DSP* алгоритме у реалном времену.

Један од најчешће коришћених *DSP* алгоритама је брза Фуријеова трансформација (*FFT*), која се користи за анализу фреквенцијских компоненти сигнала. *Arduino* има уграђене библиотеке које олакшавају извођење *FFT*-а на аналогним улазним сигнаlima.

Још један важан *DSP* алгоритам је дигитални филтер. Дигитални филтери се могу користити за уклањање шума из сигнала или за издвајање одређених фреквенција из њега. *Arduino* има неколико уграђених библиотека за имплементацију дигиталних филтера, укључујући нископропусне, високопропусне и пропусне филтере.

Поред ових основних *DSP* алгоритама, *Arduino* се такође може користити за имплементацију напреднијих техника као што су адаптивно филтрирање и таласне трансформације. Адаптивно филтрирање је техника која омогућава филтеру да прилагоди своје параметре на основу промена у улазном сигналу, док се таласне трансформације користе за анализу нестационарних сигнала као што су говор и музика.

Све у свему, *Arduino* је моћан алат за имплементацију *DSP* алгоритама у апликацијама у реалном времену. Комбиновањем флексибилности софтвера са брзином и поузданошћу хардвера, могуће је креирати софистициране *DSP* системе који се могу користити у широком спектру апликација, од аудио обраде до биомедицинске анализе сигнала.

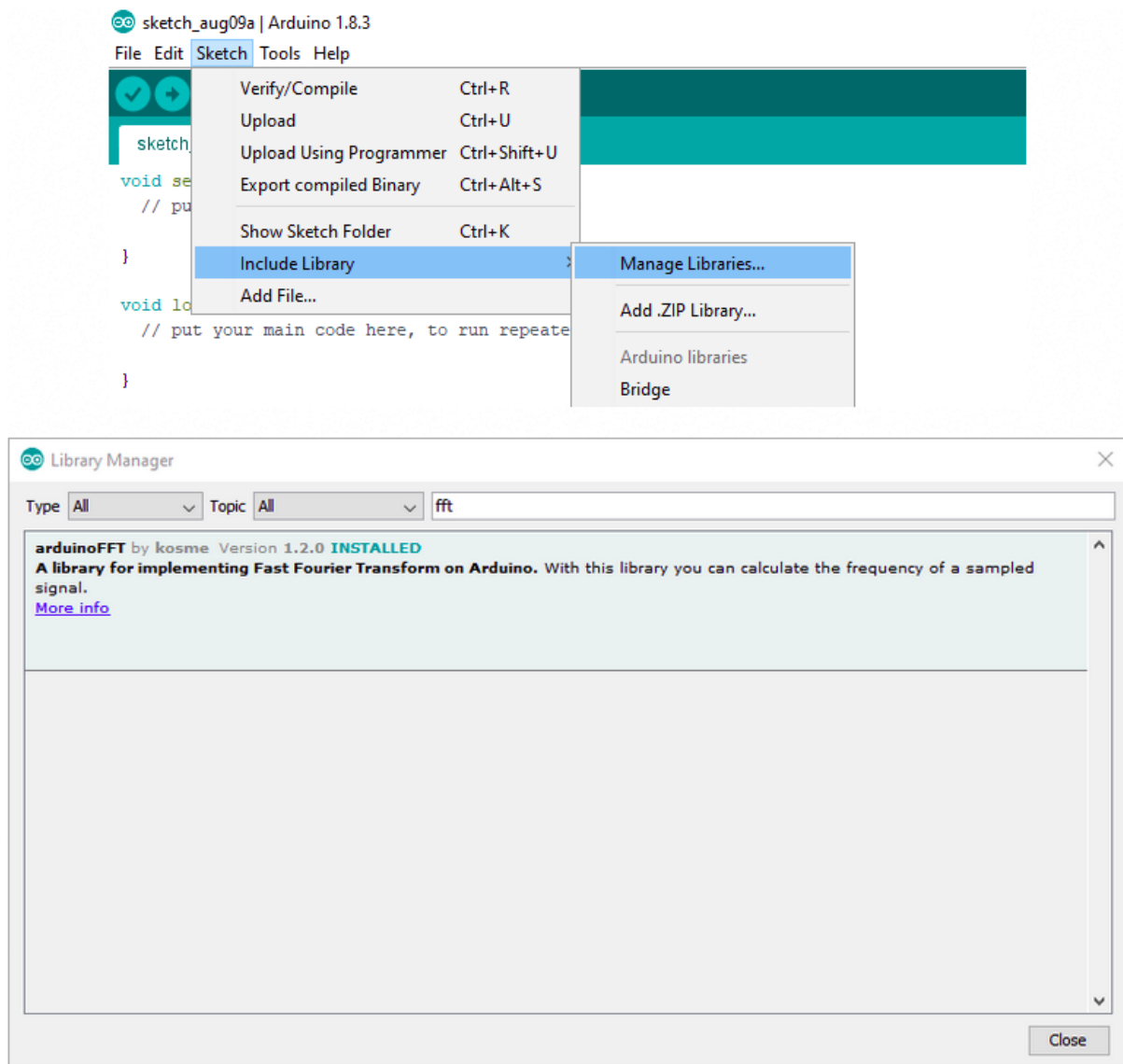
5.3. *FFT* алгоритам

Брза Фуријеова трансформација (*FFT*) је алгоритам који израчунава дискретну Фуријеову трансформацију (*DFT*) секвенце или њену инверзну (*IDFT*). Фуријеова анализа конвертује сигнал из његовог оригиналног домена (често времена или простора) у репрезентацију у фреквенцијском домену и обрнуто. *DFT* се добија декомпозицијом низа вредности на компоненте различитих фреквенција.

Овде је важан принцип да се сваки сигнал (чак и непериодични) може прилично прецизно реконструисати додавањем синусоидних сигнала заједно са различитим фреквенцијама и амплитудама. Што више синусоидних сигнала саберемо, то ће наш реконструисани сигнал више изгледати као оригинални. Теоретски гледано, са бесконачном количином синусоидних сигнала добијамо идентичан сигнал као оригинал.

Наравно, постоји неколико начина за имплементацију *FFT*-а на *Arduinu*. Можете га имплементирати од нуле или можете користити унапред направљену библиотеку. У овом раду урадићемо друго.

Да бисте инсталирали *arduinoFFT*, отворите прозор Управљање библиотекама (енг. *Manage Libraries*), унесите *fft* у траку за претрагу и инсталирајте најновију верзију.



Слика 5.2 инсталација FFT библиотеке

Подешавање које ћемо овде користити је *Arduino UNO* и генератор сигнала. Једине жице су две од генератора сигнала где једна иде на A0 пину на *Arduino*, а друга на *GND* пину. Наш сигнал има амплитуду и померања тако да скоро обухвата комплетан опсег 0-5 V, што добро одговара својствима нашег *ADC*-а.

За разлику од примера кода који долази са библиотеком, ми примењујемо *FFT* на одговарајући аналогни сигнал. Пример кода генерише симулирани синусоидни сигнал и на то примењује *FFT*.

```
#include "arduinoFFT.h"

#define SAMPLES 128 // mora biti stepen broja 2
#define SAMPLING_FREQUENCY 1000 //Hz, mora biti manje od 10000 zbog ADC

arduinoFFT FFT = arduinoFFT();

unsigned int sampling_period_us;
unsigned long microseconds;
```

```

double vReal[SAMPLES];
double vImag[SAMPLES];

void setup() {
    Serial.begin(115200);

    sampling_period_us = round(1000000 * (1.0 / SAMPLING_FREQUENCY));
}

void loop() {

    /* SAMPLING */
    for (int i=0; i<SAMPLES; i++)
    {
        microseconds = micros(); // preliva nakon 70 minuta!

        vReal[i] = analogRead(0);
        vImag[i] = 0;

        while(micros() < (microseconds + sampling_period_us)){
        }

        /* FFT */
        FFT.Windowing(vReal, SAMPLES, FFT_WIN_TYP_HAMMING, FFT_FORWARD);
        FFT.Compute(vReal, vImag, SAMPLES, FFT_FORWARD);
        FFT.ComplexToMagnitude(vReal, vImag, SAMPLES);
        double peak = FFT.MajorPeak(vReal, SAMPLES, SAMPLING_FREQUENCY);

        /* PRINT RESULTS */
        // Serial.println(peak); // štampa koje frekvencije su najdominantnije

        for(int i=0; i<(SAMPLES/2); i++)
        {
            // Serial.print((i * 1.0 * SAMPLING_FREQUENCY) / SAMPLES, 1);
            // Serial.print(" ");
            Serial.println(vReal[i], 1);
        }

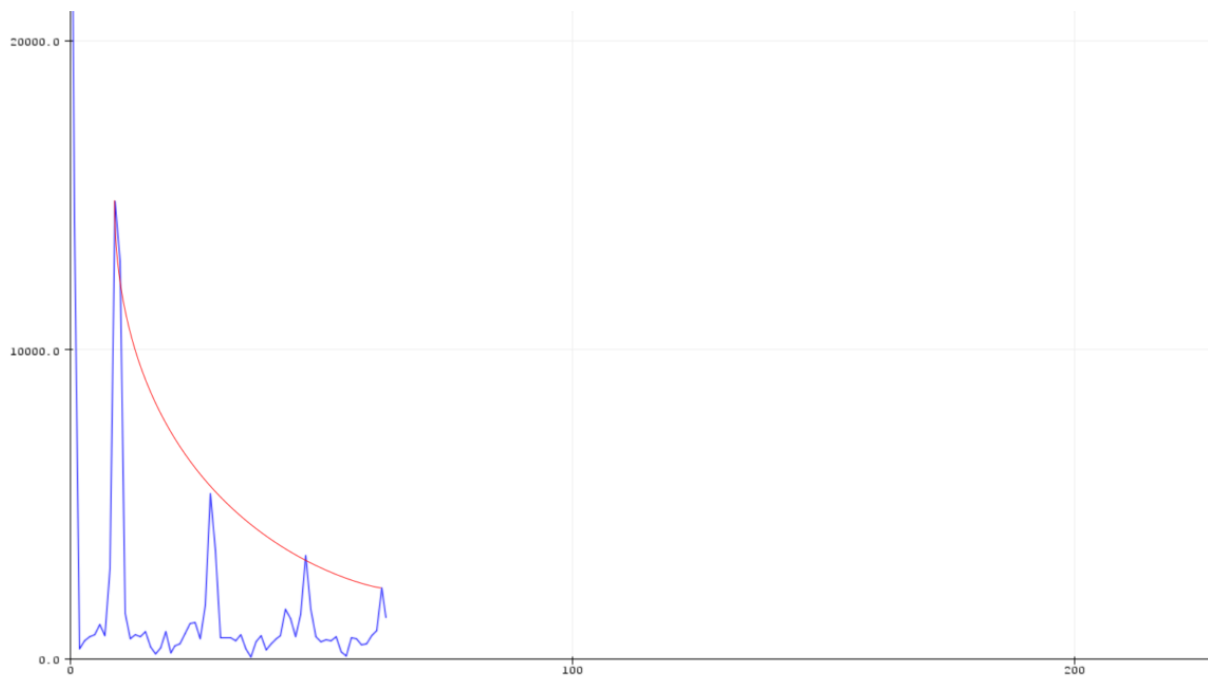
        //delay(1000); // ponovi proces svake sekunde, ili
        while(1); // pokreni kod jednom
    }
}

```

Блок кода 5.1 Пример имплементације FFT алгоритма за Arduino

Резултати мерења представљају следећи графикони који се преузимају са *Arduino* серијског плотера након покретања *FFT*-а на сигналу са стопом узорковања од 128 Hz и 128 узорака.

Бројеви на *x-оси* у графиконима испод нису фреквенција, већ број елемента. На графиконима испод, елемент број 64 је горња корпа (~500 Hz). Пошто наш улазни сигнал има *DC* офсет, добијамо врх на нули у резултатима.



Слика 5.3 FFT резултати сигнала квадратног таласа од ~ 75 Hz.

Као што видимо по црвеној линији, имамо експоненцијално опадајући тренд који добијамо за сигнал квадратног таласа.

FFT је веома згодан алат за обраду сигнала у многим областима примене. *arduinoFFT* библиотека ради сав тежак посао за вас. Међутим, коришћењем *Arduina*, прилично сте ограничени у погледу меморије и брзине узорковања. Такође је уобичајено да се покреће нископропусни филтер заједно са *FFT* алгоритмом, што нисмо урадили у нашем примеру.

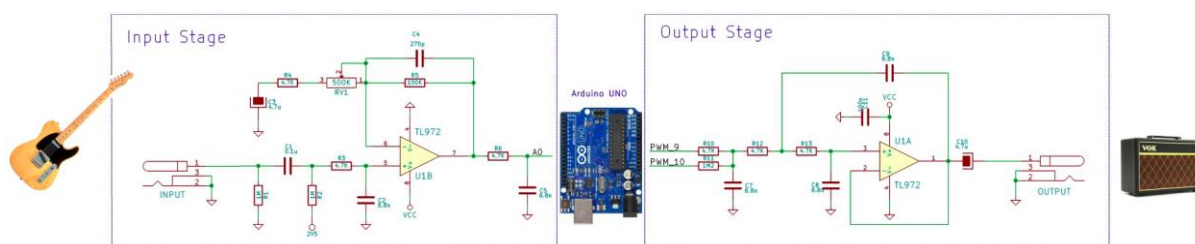
6. Програмабилна гитарска педала са *Arduinom*

Да бисмо направили програмабилну педалу за гитару са *Arduinom*, мораћемо да користимо комбинацију хардвера и софтвера. Што се тиче хардвера, користићемо стандардне електронске компоненте као што су отпорници, кондензатори и *LED* диоде за креирање кола потребних за гитарске ефекте. Такође ће нам требати матична плоча или *PCB* (штампана плоча) за израду прототипа, улазни прикључак за гитару, 9-волтна батерија и излазни прикључак за повезивање са појачалом.

На софтверској страни ћемо написати код у *C++* који ће обрадити аудио сигнал са гитаре и применити жељене ефекте. Користићемо *Arduino* развојно окружење за писање, отпремање и отклањање грешака у нашем коду.

Први корак у изградњи програмабилне гитарске педале са *Arduinom* је дизајнирање кола потребних за снимање жељеног гитарског ефекта. Ово укључује одабир одговарајућих електронских компоненти и њихово распоређивање у исправну конфигурацију на матичној плочи или штампаној плочи.

Када је коло постављено, можемо почети да пишемо софтвер у *C++* који ће обрадити аудио сигнал и применити жељени ефекат. Ово укључује коришћење уграђених функција *Arduino* платформе за читање долазног аудио сигнала, модификовање у складу са жељеним ефектом, а затим емитовање модификованог сигнала у појачало.



Слика 6.1 шема улазно излазних кола *Arduino UNO* гитарске педале

Како ово коло са слике (6.1) ради? Штит за *Arduino* има три дела:

Улазна фаза: Појачава и филтрира гитарски сигнал чинећи га спремним за *Arduino UNO* аналогно дигитални претварач (ADC).

***Arduino* плоча:** преузима дигитализовани таласни облик од ADC-а и обавља сву дигиталну обраду сигнала (*DSP*) стварајући ефекте (изобличење, кашњење, јачина звука, амбијент...).

Излазна фаза: Када се креира нови таласни облик, сигнал се узима са *Arduino* дигиталних излаза (комбинована два *PWM*-а) и припрема се за слање следећој педали или гитарском појачалу.

Како програмирати *Arduino* ако желимо да постигнемо ефекат гитарске педале која се зове clean boost што значи чисто појачање?

```
#define LED 13
#define FOOTSWITCH 12
#define TOGGLE 2
```

```

#define PUSHBUTTON_1 A5
#define PUSHBUTTON_2 A4
#define PWM_FREQ 0x00FF
#define PWM_MODE 0
#define PWM_QTY 2

int input, vol_variable = 10000;
int counter = 0;
unsigned int ADC_low, ADC_high;

void setup() {
    pinMode(FOOTSWITCH, INPUT_PULLUP);
    pinMode(TOGGLE, INPUT_PULLUP);
    pinMode(PUSHBUTTON_1, INPUT_PULLUP);
    pinMode(PUSHBUTTON_2, INPUT_PULLUP);
    pinMode(LED, OUTPUT);

    ADMUX = 0x60;
    ADCSRA = 0xe5;
    ADCSRB = 0x07;
    DIDR0 = 0x01;

    TCCR1A = (((PWM_QTY - 1) << 5) | 0x80 | (PWM_MODE << 1));
    TCCR1B = ((PWM_MODE << 3) | 0x11);
    TIMSK1 = 0x20;
    ICR1H = (PWM_FREQ >> 8);
    ICR1L = (PWM_FREQ & 0xff);
    DDRB |= ((PWM_QTY << 1) | 0x02);
    sei();
}

void loop() {
    if (digitalRead(FOOTSWITCH)) digitalWrite(LED, HIGH);
    else digitalWrite(LED, LOW);
}

ISR(TIMER1_CAPT_vect) {
    ADC_low = ADCL;
    ADC_high = ADCH;
    input = ((ADC_high << 8) | ADC_low) + 0x8000;
    counter++;
    if (counter == 100) {
        counter = 0;
        if (!digitalRead(PUSHBUTTON_2)) {
            if (vol_variable < 32768) vol_variable = vol_variable + 10; //
pojačati signal
            digitalWrite(LED, LOW);
        }

        if (!digitalRead(PUSHBUTTON_1)) {
            if (vol_variable > 0) vol_variable = vol_variable - 10;
            digitalWrite(LED, LOW);
        }
    }

    input = map(input, -32768, +32768, -vol_variable, vol_variable);
    OCR1AL = ((input + 0x8000) >> 8);
}

```

```
OCR1BL = input;  
}
```

Блок кода 6.1 Пример имплементације FFT алгоритма за Arduino

Програм који нам омогућава да улазни сигнал који доводимо на *Arduino* кроз улазну фазу, појачавамо тако да не долази до искривљења, илити дисторзије сигнала, погледати блок кода 6.1.

7. Закључак

Експериментисање са гитарском музиком, данас је постало веома активно подручје иновација и истраживања, поготово популарношћу микроконтролера који нам омогућавају да са мало и са основним техничким знањем можемо ући у свет обраде аудио сигнала.

Овај рад је истраживао употребу програмског језика C++ за креирање прилагођених педала за гитарске ефекте помоћу *Arduino* микроконтролера. Разговарали смо о основама језика C++ и *Arduino* платформи, затим о основама *DSP*-а имплементацију и мерења *FFT* алгоритма, а онда смо се упустили у то како направити и програмирати прилагођене ефекте гитаре користећи C++ на *Arduinu*. Такође смо истражили хардвер и софтвер потребан за прављење програмабилне гитарске педале са *Arduinom*.

Овај пројекат је само почетак онога што се може постићи са C++ језиком и *Arduino* платформом. Постоје многе друге врсте пројеката који се могу изградити помоћу ових алата, укључујући системе кућне аутоматизације, пројекте роботике и још много тога. Са неким основним знањем о електроници и програмирању, свако може да почне да прави сопствене пројекте користећи C++ и *Arduino*.

8. Литература

- [1] Laslo Kraus, Programski jezik C++ sa rešenim zadacima, 11. izdanje, Beograd 2019.
- [2] C++ Introduction (https://www.w3schools.com/cpp/cpp_intro.asp)
- [3] Arduino Documentation (<https://docs.arduino.cc/>)
- [4] Arduino Hardware (<https://www.arduino.cc/en/hardware>)
- [5] Arduino Software (<https://www.arduino.cc/en/software>)
- [6] Arduino Uno Rev3 (<https://store.arduino.cc/products/arduino-uno-rev3>)
- [7] Arduino Shields (<https://learn.sparkfun.com/tutorials/arduino-shields>)
- [8] What is an Arduino? (<https://learn.sparkfun.com/tutorials/what-is-an-arduino/whats-on-the-board>)
- [9] Introduction To Guitar Effect Pedals (<https://www.libertyparkmusic.com/guitar-effect-pedals-types-and-sounds/>)
- [10] A Beginner's Guide to Digital Signal Processing (DSP) (<https://www.analog.com/en/design-center/landing-pages/001/beginners-guide-to-dsp.html>)
- [11] Basic Signal Processing (<https://www.ssla.co.uk/basic-signal-processing/>)
- [12] Fast Fourier Transformation FFT - Basics (<https://www.nti-audio.com/en/support/know-how/fast-fourier-transform-fft>)
- [13] What Is FFT and How Can You Implement It on an Arduino? (<https://www.norwegiancreations.com/2017/08/what-is-fft-and-how-can-you-implement-it-on-an-arduino>)
- [14] pedalSHIELD UNO Arduino Guitar Pedal (<https://www.electrosmash.com/pedalshield-uno>)
- [15] ElectroSmash / pedalshield github (<https://github.com/ElectroSmash/pedalshield>)